Universidad Nacional

Sede Regional Brunca – Campus Coto

BlaxtorGym

Plan de desarrollo modular

Arquitectura, dependencias y buenas prácticas

Integrantes

Justin Rojas Jarquin
Cristopher Ureña Valverde

Tecnologías base

C# • WPF • .NET • SQLite • Entity Framework Core

Primero funcionalidad, persistencia y estabilidad; después diseño visual.

Objetivo del plan

Este documento define un orden de implementación claro para BlaxtorGym. Cada módulo indica su responsable, sus dependencias y el resultado mínimo que debe quedar terminado antes de continuar.

Idea clave

El número de fase representa una **dependencia lógica**, no una fila de espera. Si dos módulos no dependen entre sí, Justin y Cristopher pueden desarrollarlos en paralelo.

Orden cronológico de desarrollo

Para facilitar la lectura, esta tabla se presenta en orientación horizontal. Cada fila define claramente quién trabaja el módulo, qué debe existir antes de iniciarlo y cuál es el resultado mínimo esperado.

Fase	Responsable	Módulo / etapa	Dependencias para iniciar	Resultado mínimo de la fase
1	Justin	Configuración inicial del proyecto	No posee dependencias. Es el punto de inicio del desarrollo.	Solución creada, estructura de carpetas definida, paquetes NuGet necesarios, configuración base y convenciones de código establecidas.
2	Justin	Persistencia: SQLite + EF Core	Requiere la **Fase 1**.	`AppDbContext`, configuración de SQLite, migraciones, datos iniciales (seeding) y persistencia base funcionando correctamente.
3	Justin	Autenticación / Login	Requiere la **Fase 2** y la definición de la entidad `Usuario`.	Inicio de sesión funcional con BCrypt, validación de credenciales, usuarios activos/inactivos y control de intentos fallidos.
4	Justin	Tipos de cliente	Requiere la **Fase 2**.	CRUD y reglas de negocio de tipos de cliente funcionando. Al finalizar esta fase se desbloquea el desarrollo del módulo Clientes.
5	Cristopher	Clientes	Requiere las **Fases 2 y 4**: persistencia y tipos de cliente.	Registro, consulta, modificación, desactivación y validaciones de clientes. Este módulo desbloquea Mediciones y participa en Membresías.
6	Justin	Tipos de membresía + Tarifas	Requiere las **Fases 2 y 4**. Puede desarrollarse en paralelo con Clientes.	Tipos de membresía, duración, precios, combinaciones por tipo de cliente y vigencia de tarifas correctamente implementados.
7	Cristopher	Mediciones físicas	Requiere la **Fase 5**: Clientes.	Registro e historial de mediciones físicas por cliente, con los cálculos y validaciones definidos para el módulo.
8	Justin	Membresías	Requiere las **Fases 5 y 6**: Clientes y Tarifas/Tipos de membresía.	Creación y renovación de membresías, fechas de vigencia, vencimiento y estado correctamente administrados.
9	Justin	Pagos	Requiere la **Fase 8**: Membresías.	Registro e historial de pagos correctamente asociados a las membresías correspondientes.
10	Cristopher	Notificaciones	Requiere las **Fases 5 y 8**: Clientes y Membresías.	Generación de avisos relacionados con vencimientos y control del estado de envío de cada notificación.
11	Cristopher	Backups	Requiere la **Fase 2**. Puede adelantarse una vez que la estructura de la base de datos sea estable.	Respaldo y restauración controlada de SQLite sin bloquear la interfaz ni comprometer la integridad de los datos.
12	Cristopher	Reportes / PDF	Requiere las **Fases 5, 7, 8 y 9**: Clientes, Mediciones, Membresías y Pagos.	Reportes construidos a partir de datos reales y generación de PDF desacoplada de la interfaz gráfica.
13	Ambos	Bitácora de cambios	Requiere que los módulos principales ya sean funcionales.	Registro consistente de acciones y cambios relevantes realizados dentro del sistema.
14	Ambos	Integración general	Requiere las **Fases 1–13**.	Navegación completa, inyección de dependencias, permisos, integración entre módulos y manejo uniforme de errores.

Fase	Responsable	Módulo / etapa	Dependencias para iniciar	Resultado mínimo de la fase
15	Ambos	Diseño visual definitivo	Requiere que el sistema esté funcional e integrado.	Aplicación de estilos, recursos visuales y mejoras de experiencia de usuario sin modificar las reglas de negocio ya validadas.
16	Ambos	Pruebas y optimización final	Requiere todas las fases anteriores.	Pruebas funcionales, regresión, pruebas con crecimiento de datos, medición de rendimiento y corrección de cuellos de botella.

Ruta crítica

Configuración → **Persistencia** → **Tipos de cliente** → **Clientes + Tarifas** → **Membresías** → **Pagos** → **Integración**.
Reportes / PDF se desarrolla cuando ya existan datos reales de Clientes, Mediciones, Membresías y Pagos. **Backups** puede adelantarse desde que la base de datos sea estable.

Distribución por integrante

Justin	Cristopher
Configuración inicial	
SQLite + EF Core	
Autenticación / Login	
Tipos de cliente	
Tipos de membresía + Tarifas	
Membresías	
Pagos	Clientes
Mediciones físicas	
Notificaciones	
Backups	
Reportes / PDF	

Trabajo conjunto: Bitácora, integración, diseño visual, pruebas y optimización.

Principios obligatorios de desarrollo

SOLID	Cada clase debe tener una responsabilidad clara. Usar interfaces e inyección de dependencias y evitar servicios gigantes o acoplamiento directo entre WPF e Infrastructure.
Clean Code	Nombres explícitos, métodos pequeños, validaciones centralizadas, poca duplicación y nada de lógica de negocio dentro del code-behind.
MVVM	La View presenta; el ViewModel maneja estado/comandos; Application Service ejecuta el caso de uso; Repository persiste. La View nunca consulta SQLite directamente.
Async/Await	Base de datos, correo, PDF y backups deben ser asíncronos cuando corresponda. Evitar `.Result` y `.Wait()` en el hilo de interfaz.
Errores	Excepciones controladas, mensajes comprensibles para el usuario y logging técnico. No usar `catch` vacíos.

Reglas para mantener la fluidez

SOLID y Clean Code mejoran la mantenibilidad, pero **no garantizan rendimiento por sí solos**. La fluidez se protege con decisiones concretas:

1. **Consultar solo lo necesario.** Filtrar, ordenar y proyectar desde EF Core/SQLite; no traer tablas completas para filtrarlas luego en C#.
2. **Usar `AsNoTracking()`** en consultas de solo lectura.
3. **Paginar o cargar incrementalmente** listados grandes de clientes, pagos, membresías y bitácora.
4. **Virtualizar controles WPF** como `DataGrid` o `ListView` cuando existan muchas filas.
5. **Crear índices SQLite** para campos realmente utilizados en búsquedas, relaciones y filtros frecuentes.
6. **Evitar el problema N+1**: no ejecutar una consulta adicional por cada elemento mostrado.
7. **No bloquear el hilo de UI**. PDF, correo, backup y operaciones de I/O no deben congelar la ventana.
8. **No mantener colecciones enormes en memoria** si no son necesarias para la pantalla actual.
9. **Medir antes de optimizar**. Registrar operaciones lentas y probar el sistema con volúmenes crecientes de datos.

Meta de rendimiento

La meta no debe redactarse como “cero lag”, porque ningún sistema serio puede garantizarlo de forma absoluta. La meta técnica será mantener una **interfaz perceptiblemente responsiva** y evitar degradaciones importantes conforme aumente la cantidad de datos, validándolo mediante pruebas y mediciones.

Criterio de terminado por módulo

Definition of Done técnico

Un módulo no está terminado solo porque abre una pantalla. Debe cumplir sus reglas de negocio, persistir correctamente, validar entradas, manejar errores, no bloquear la interfaz, respetar la arquitectura, pasar sus pruebas principales y quedar integrado sin romper funcionalidades anteriores.